# IT 2080 – IT Project

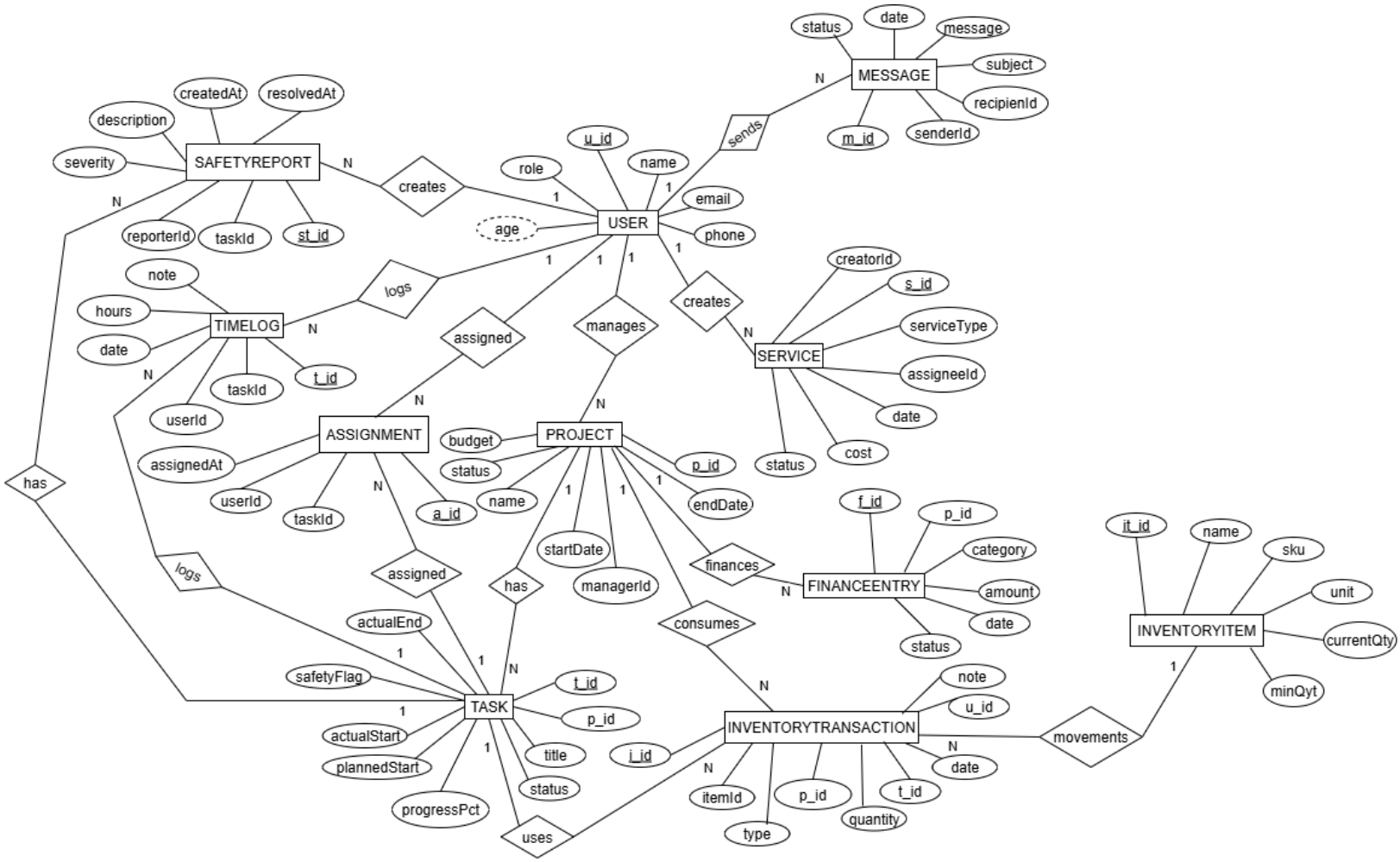
## Activity 04


**Group Number: ITP_B2_W227**

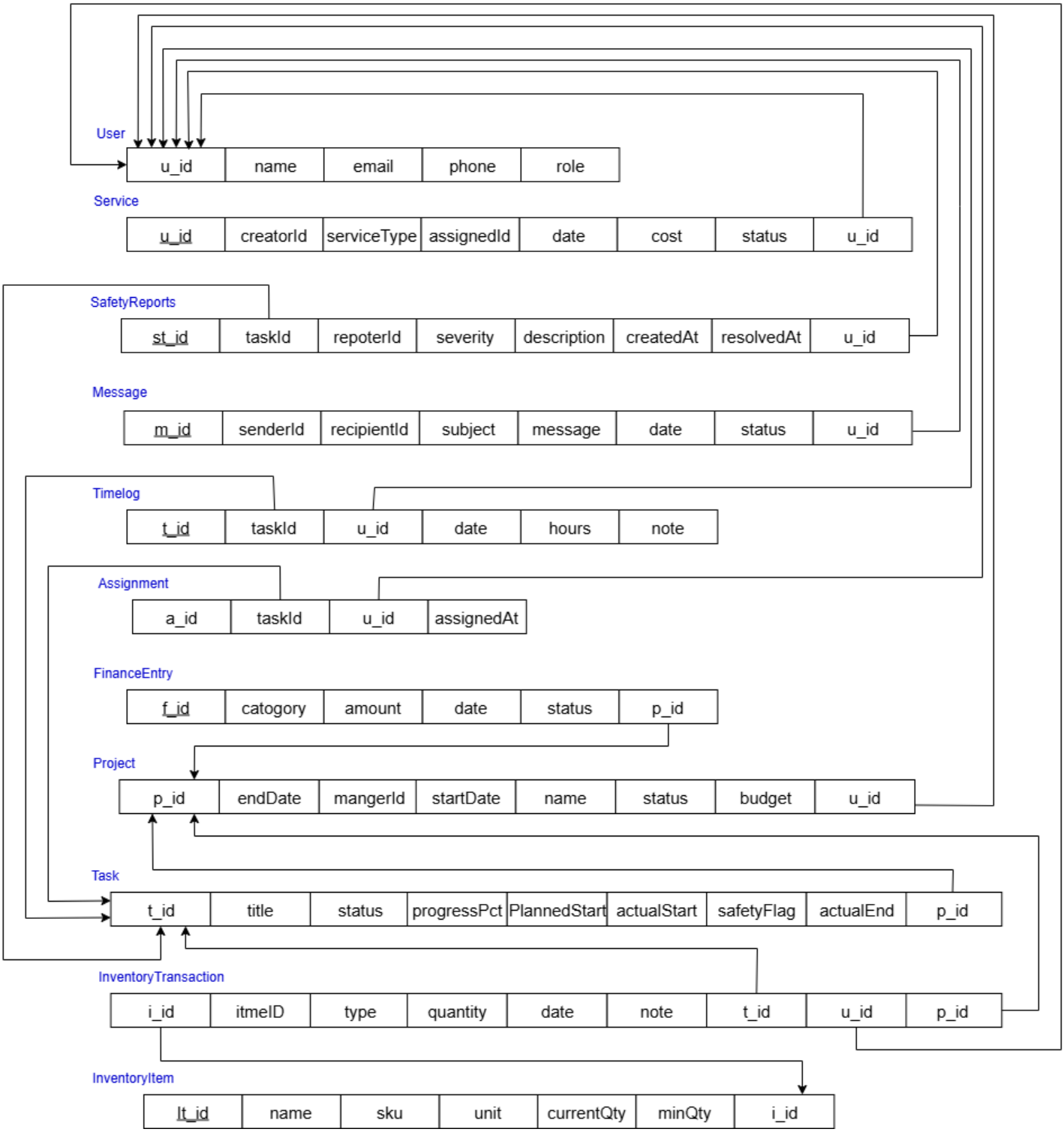
**Topic: Construction Management System**

| | Registration Number | Student Name | Email address | Contact no |
|---|---|---|---|---|
| 1 | IT23615502 | J.A.A.D. Jayakodi | it23615502@my.sliit.lk | 0701038400 |
| 2 | IT23562424 | M.G.C.S. Bandara | it23562424@my.sliit.lk | 0787911287 |
| 3 | IT23698772 | D.D.T. Osadara | it23698772@my.sliit.lk | 0725556628 |
| 4 | IT23561816 | H.A.B.P. Athukorala | it23561816@my.sliit.lk | 0775550410 |
| 5 | IT23584372 | H.G.P.N.Chandrarathna | it23584372@my.sliit.lk | 0701622626 |

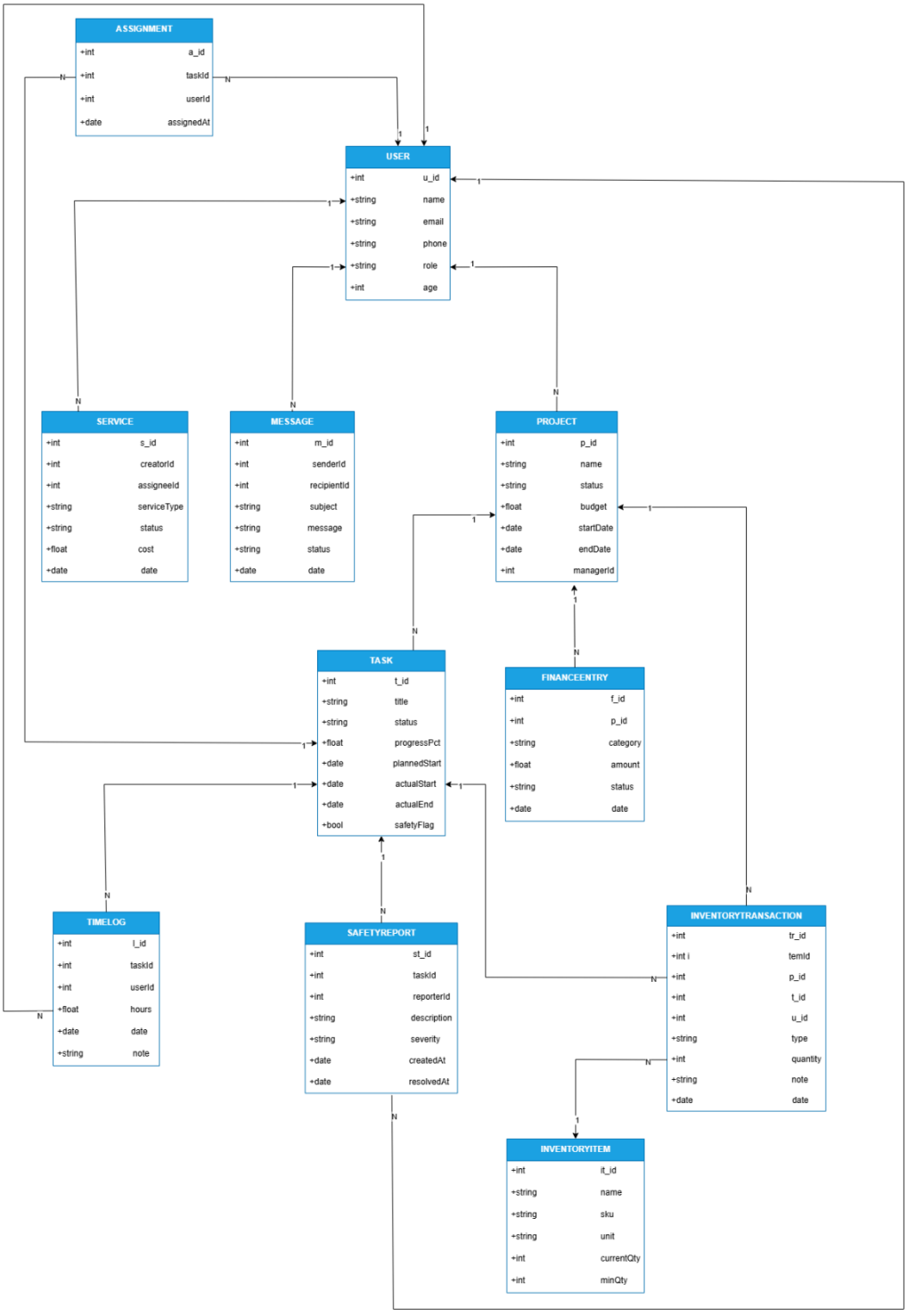
**01. Design a comprehensive ER diagram to represent the data model, capturing all entities, attributes, and relationships.**



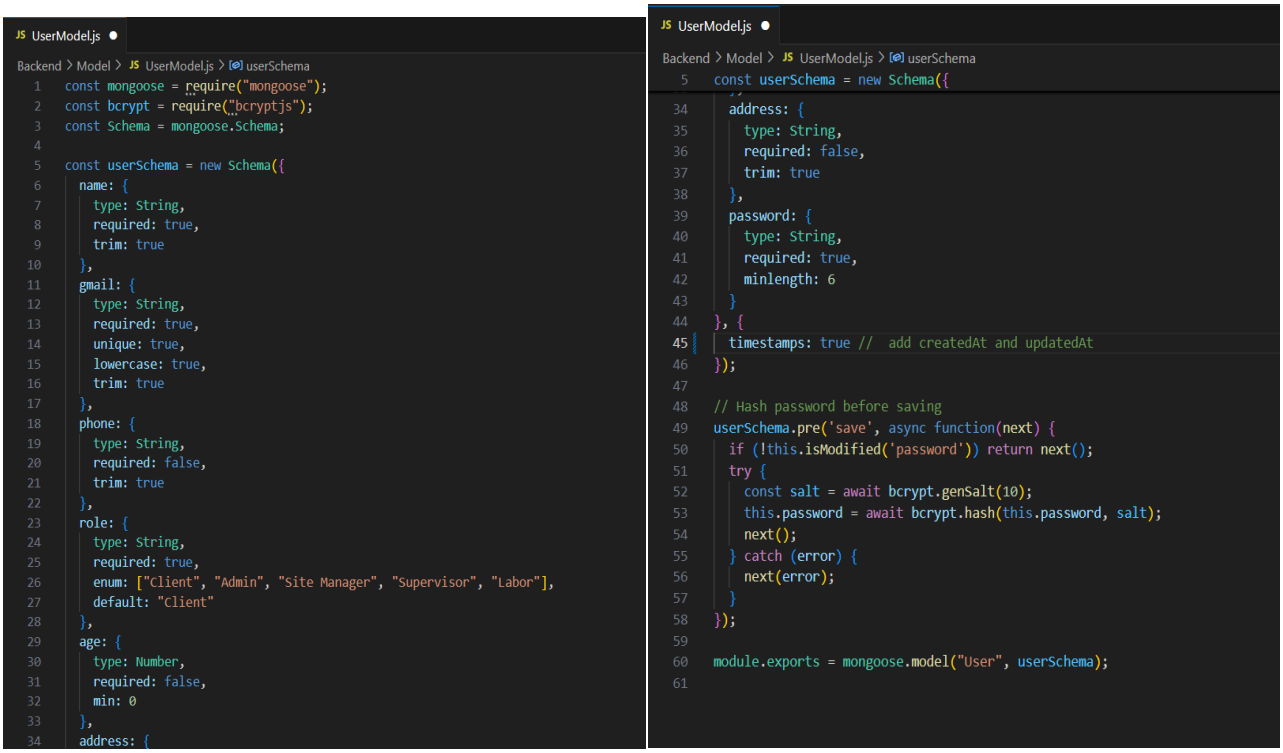
**2. Normalize the database schema to eliminate redundancy, improve data integrity, and ensure optimal performance.**



**3. Build the database using the chosen technology, ensuring adherence to the designed schema, and incorporating all necessary constraints, indexes, and relationships.**

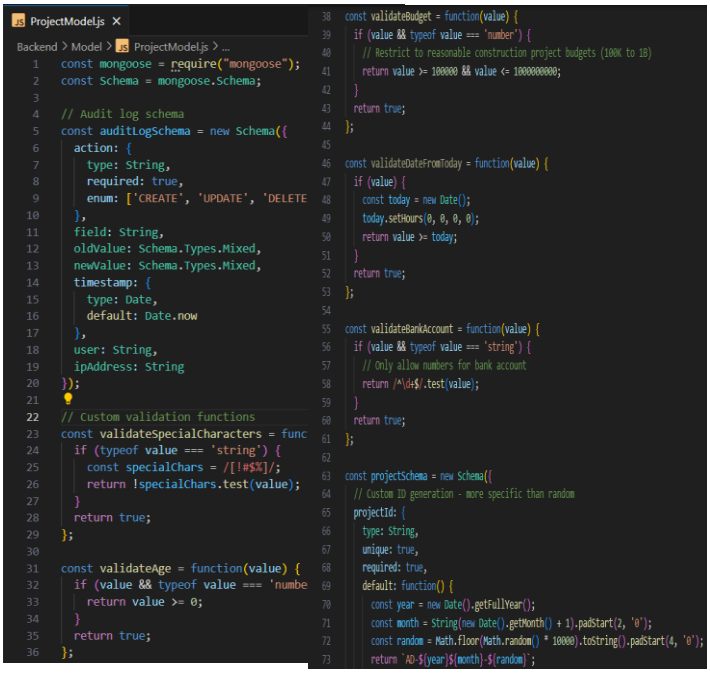


## User model



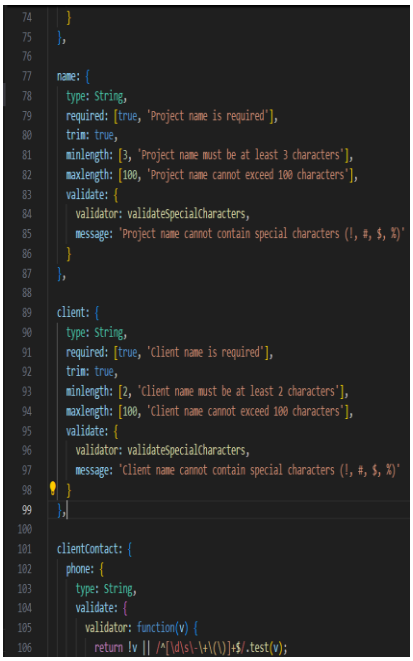
```
UserModel.js
Backend > Model > UserModel.js > userSchema
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const Schema = mongoose.Schema;

const userSchema = new Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  gmail: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  phone: {
    type: String,
    required: false,
    trim: true
  },
  role: {
    type: String,
    required: true,
    enum: ["Client", "Admin", "Site Manager", "Supervisor", "Labor"],
    default: "Client"
  },
  age: {
    type: Number,
    required: false,
    min: 0
  },
  address: {
    type: String,
    required: false,
    trim: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  }
}, {
  timestamps: true //  add createdAt and updatedAt
});

// Hash password before saving
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  try {
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
    next();
  } catch (error) {
    next(error);
  }
});

module.exports = mongoose.model("User", userSchema);
```

## Project model

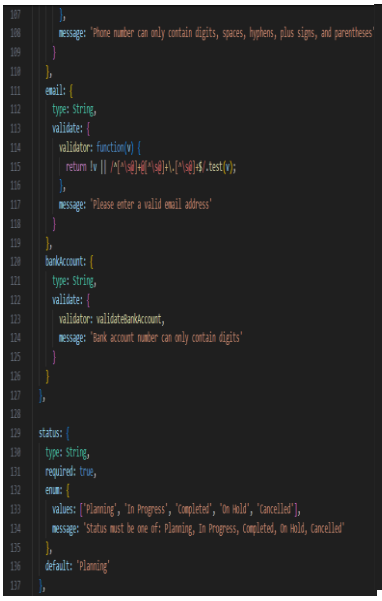


```
ProjectModel.js
Backend > Model > ProjectModel.js > ...
const mongoose = require("mongoose");
const Schema = mongoose.Schema;

// Audit log schema
const auditLogSchema = new Schema({
  action: {
    type: String,
    required: true,
    enum: ['CREATE', 'UPDATE', 'DELETE
  },
  field: String,
  oldValue: Schema.Types.Mixed,
  newValue: Schema.Types.Mixed,
  timestamp: {
    type: Date,
    default: Date.now
  },
  user: String,
  ipAddress: String
});
// Custom validation functions
const validateSpecialCharacters = func
  if (typeof value === 'string') {
    const specialChars = /[!#$%]/;
    return !specialChars.test(value);
  }
  return true;
};

const validateAge = function(value) {
  if (value && typeof value === 'numbe
    return value >= 0;
  }
  return true;
};
```

```
const validateBudget = function(value) {
  if (value && typeof value === 'number') {
    // Restrict to reasonable construction project budgets (100K to 1B)
    return value >= 100000 && value <= 1000000000;
  }
  return true;
};

const validateDateFromToday = function(value) {
  if (value) {
    const today = new Date();
    today.setHours(0, 0, 0, 0);
    return value >= today;
  }
  return true;
};

const validateBankAccount = function(value) {
  if (value && typeof value === 'string') {
    // Only allow numbers for bank account
    return /^\d+$/.test(value);
  }
  return true;
};

const projectSchema = new Schema({
  // Custom ID generation - more specific than random
  projectId: {
    type: String,
    unique: true,
    required: true,
    default: function() {
      const year = new Date().getFullYear();
      const month = String(new Date().getMonth() + 1).padStart(2, '0');
      const random = Math.floor(Math.random() * 10000).toString().padStart(4, '0');
      return `AD-${year}${month}-${random}`;
```

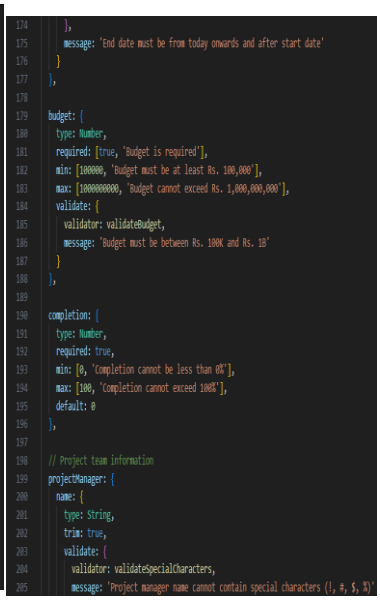


```
  },

  name: {
    type: String,
    required: [true, 'Project name is required'],
    trim: true,
    minlength: [3, 'Project name must be at least 3 characters'],
    maxlength: [100, 'Project name cannot exceed 100 characters'],
    validate: {
      validator: validateSpecialCharacters,
      message: 'Project name cannot contain special characters (!, #, $, %)'
    }
  },

  client: {
    type: String,
    required: [true, 'Client name is required'],
    trim: true,
    minlength: [2, 'Client name must be at least 2 characters'],
    maxlength: [100, 'Client name cannot exceed 100 characters'],
    validate: {
      validator: validateSpecialCharacters,
      message: 'Client name cannot contain special characters (!, #, $, %)'
    }
  },

  clientContact: {
    phone: {
      type: String,
      validate: {
        validator: function(v) {
          return !v || /^[\d\s\-\+\(\)]+$/.test(v);
```

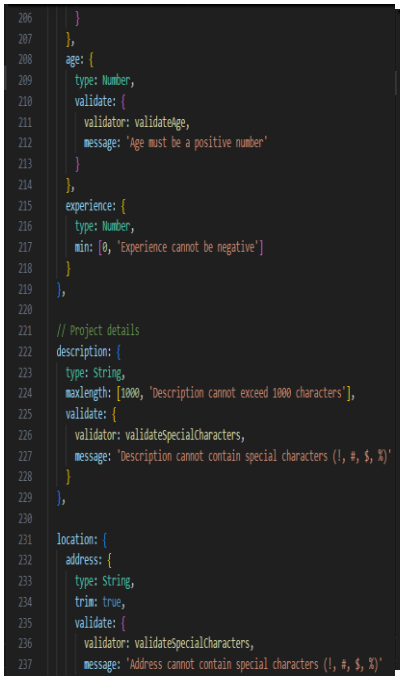


```
      },
      message: 'Phone number can only contain digits, spaces, hyphens, plus signs, and parentheses'
    }
  },
  email: {
    type: String,
    validate: {
      validator: function(v) {
        return !v || /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(v);
      },
      message: 'Please enter a valid email address'
    }
  },
  bankAccount: {
    type: String,
    validate: {
      validator: validateBankAccount,
      message: 'Bank account number can only contain digits'
    }
  }
},

status: {
  type: String,
  required: true,
  enum: {
    values: ['Planning', 'In Progress', 'Completed', 'On Hold', 'Cancelled'],
    message: 'Status must be one of: Planning, In Progress, Completed, On Hold, Cancelled'
  },
  default: 'Planning'
},
```

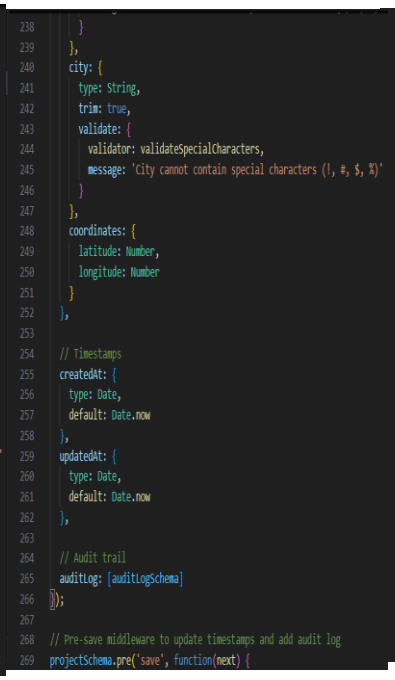
```
priority: {
  type: String,
  enum: ['Low', 'Medium', 'High', 'Critical'],
  default: 'Medium'
},

startDate: {
  type: Date,
  required: [true, 'Start date is required'],
  validate: {
    validator: function (value) {
      // Only enforce future-or-today constraint when creating a new document
      if (this.isNew) {
        const today = new Date();
        today.setHours(0, 0, 0, 0);
        return value >= today;
      }
      // On edits, allow retaining or setting past dates
      return true;
    },
    message: 'Start date must be from today onwards for new projects'
  }
},

endDate: {
  type: Date,
  required: [true, 'End date is required'],
  validate: {
    validator: function(value) {
      if (!value) return false;
      const today = new Date();
      today.setHours(0, 0, 0, 0);
      const isFromTodayOnwards = value >= today;
      if (!this.startDate) return isFromTodayOnwards;
      return isFromTodayOnwards && value > this.startDate;
```



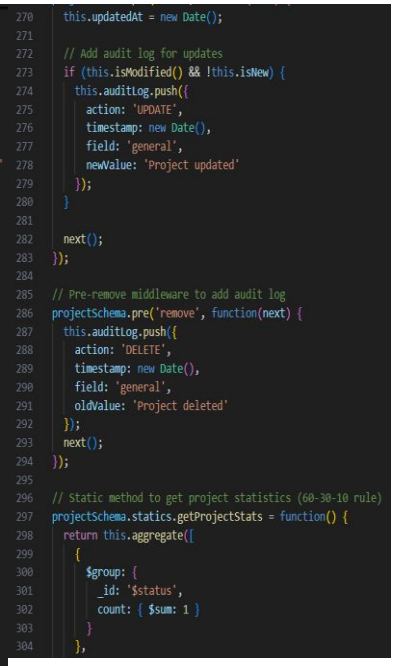
```
    },
    message: 'End date must be from today onwards and after start date'
  }
},

budget: {
  type: Number,
  required: [true, 'Budget is required'],
  min: [100000, 'Budget must be at least Rs. 100,000'],
  max: [1000000000, 'Budget cannot exceed Rs. 1,000,000,000'],
  validate: {
    validator: validateBudget,
    message: 'Budget must be between Rs. 100K and Rs. 1B'
  }
},

completion: {
  type: Number,
  required: true,
  min: [0, 'Completion cannot be less than 0%'],
  max: [100, 'Completion cannot exceed 100%'],
  default: 0
},

// Project team information
projectManager: {
  name: {
    type: String,
    trim: true,
    validate: {
      validator: validateSpecialCharacters,
      message: 'Project manager name cannot contain special characters (!, #, $, %)'
```



```
    }
  },
  age: {
    type: Number,
    validate: {
      validator: validateAge,
      message: 'Age must be a positive number'
    }
  },
  experience: {
    type: Number,
    min: [0, 'Experience cannot be negative']
  }
},

// Project details
description: {
  type: String,
  maxlength: [1000, 'Description cannot exceed 1000 characters'],
  validate: {
    validator: validateSpecialCharacters,
    message: 'Description cannot contain special characters (!, #, $, %)'
  }
},

location: {
  address: {
    type: String,
    trim: true,
    validate: {
      validator: validateSpecialCharacters,
      message: 'Address cannot contain special characters (!, #, $, %)'
```

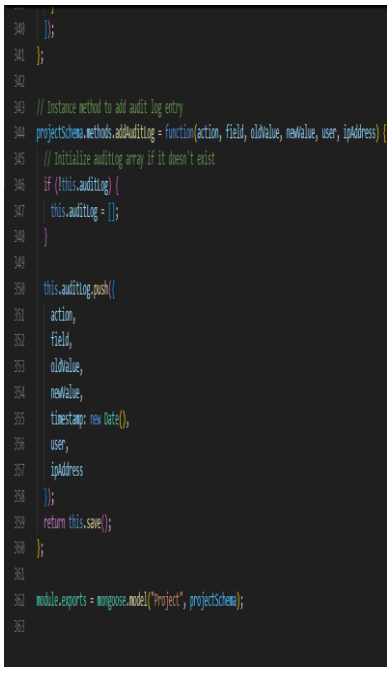


```
    }
  },
  city: {
    type: String,
    trim: true,
    validate: {
      validator: validateSpecialCharacters,
      message: 'City cannot contain special characters (!, #, $, %)'
    }
  },
  coordinates: {
    latitude: Number,
    longitude: Number
  }
},

// Timestamps
createdAt: {
  type: Date,
  default: Date.now
},
updatedAt: {
  type: Date,
  default: Date.now
},

// Audit trail
auditLog: [auditLogSchema]
});

// Pre-save middleware to update timestamps and add audit log
projectSchema.pre('save', function(next) {
```



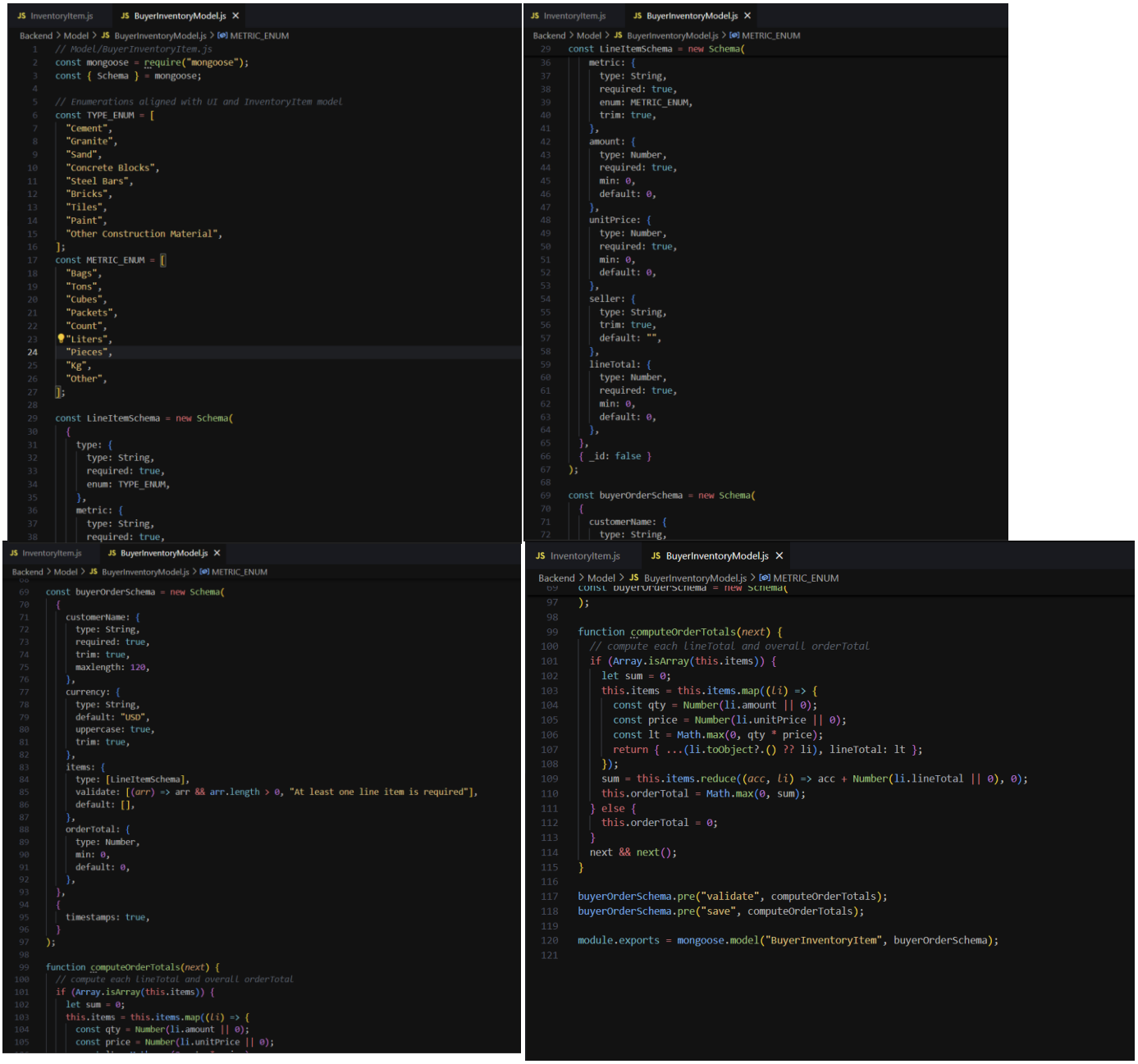
```
  this.updatedAt = new Date();

  // Add audit log for updates
  if (this.isModified() && !this.isNew) {
    this.auditLog.push({
      action: 'UPDATE',
      timestamp: new Date(),
      field: 'general',
      newValue: 'Project updated'
    });
  }

  next();
});

// Pre-remove middleware to add audit log
projectSchema.pre('remove', function(next) {
  this.auditLog.push({
    action: 'DELETE',
    timestamp: new Date(),
    field: 'general',
    oldValue: 'Project deleted'
  });
  next();
});

// Static method to get project statistics (60-30-10 rule)
projectSchema.statics.getProjectStats = function() {
  return this.aggregate([
    {
      $group: {
        _id: '$status',
        count: { $sum: 1 }
      }
    },
```

```
    {
      $group: {
        _id: null,
        total: { $sum: '$count' },
        statusCounts: {
          $push: {
            status: '$_id',
            count: '$count'
          }
        }
      }
    },
    {
      $project: {
        _id: 0,
        total: 1,
        statusCounts: 1,
        distribution: {
          $map: {
            input: '$statusCounts',
            as: 'status',
            in: {
              status: '$$status.status',
              count: '$$status.count',
              percentage: {
                $multiply: [
                  { $divide: ['$$status.count', '$total'] },
                  100
                ]
              }
            }
          }
        }
      }
    }
  ]);
};
```



```
  ]);
};

// Instance method to add audit log entry
projectSchema.methods.addAuditLog = function(action, field, oldValue, newValue, user, ipAddress) {
  // Initialize auditLog array if it doesn't exist
  if (!this.auditLog) {
    this.auditLog = [];
  }

  this.auditLog.push({
    action,
    field,
    oldValue,
    newValue,
    timestamp: new Date(),
    user,
    ipAddress
  });
  return this.save();
};

module.exports = mongoose.model("Project", projectSchema);
```

## Inventory model



```
// Model/BuyerInventoryItem.js
const mongoose = require("mongoose");
const { Schema } = mongoose;

// Enumerations aligned with UI and InventoryItem model
const TYPE_ENUM = [
  "Cement",
  "Granite",
  "Sand",
  "Concrete Blocks",
  "Steel Bars",
  "Bricks",
  "Tiles",
  "Paint",
  "Other Construction Material",
];
const METRIC_ENUM = [
  "Bags",
  "Tons",
  "Cubes",
  "Packets",
  "Count",
  "Liters",
  "Pieces",
  "Kg",
  "Other",
];

const LineItemSchema = new Schema(
  {
    type: {
      type: String,
      required: true,
      enum: TYPE_ENUM,
    },
    metric: {
      type: String,
      required: true,
      enum: METRIC_ENUM,
      trim: true,
    },
    amount: {
      type: Number,
      required: true,
      min: 0,
      default: 0,
    },
    unitPrice: {
      type: Number,
      required: true,
      min: 0,
      default: 0,
    },
    seller: {
      type: String,
      trim: true,
      default: "",
    },
    lineTotal: {
      type: Number,
      required: true,
      min: 0,
      default: 0,
    },
  },
  { _id: false }
);

const buyerOrderSchema = new Schema(
  {
    customerName: {
      type: String,
      required: true,
      trim: true,
      maxlength: 120,
    },
    currency: {
      type: String,
      default: "USD",
      uppercase: true,
      trim: true,
    },
    items: {
      type: [LineItemSchema],
      validate: [(arr) => arr && arr.length > 0, "At least one line item is required"],
      default: [],
    },
    orderTotal: {
      type: Number,
      min: 0,
      default: 0,
    },
  },
  {
    timestamps: true,
  }
);

function computeOrderTotals(next) {
  // compute each lineTotal and overall orderTotal
  if (Array.isArray(this.items)) {
    let sum = 0;
    this.items = this.items.map((li) => {
      const qty = Number(li.amount || 0);
      const price = Number(li.unitPrice || 0);
      const lt = Math.max(0, qty * price);
      return { ...(li.toObject?.() ?? li), lineTotal: lt };
    });
    sum = this.items.reduce((acc, li) => acc + Number(li.lineTotal || 0), 0);
    this.orderTotal = Math.max(0, sum);
  } else {
    this.orderTotal = 0;
  }
  next && next();
}

buyerOrderSchema.pre("validate", computeOrderTotals);
buyerOrderSchema.pre("save", computeOrderTotals);

module.exports = mongoose.model("BuyerInventoryItem", buyerOrderSchema);
```

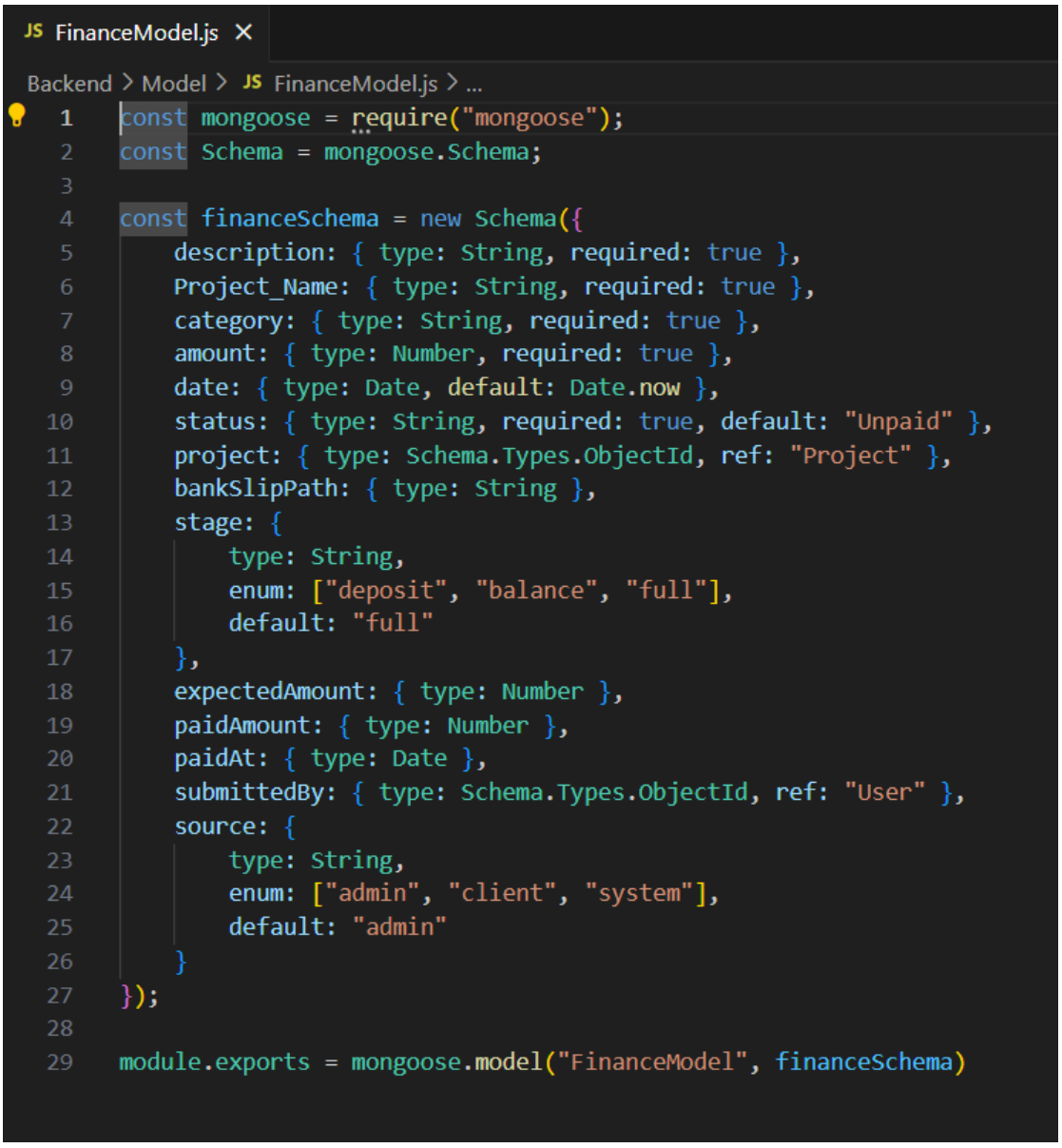
## Communication model

ServiceRoute.js M | MessageModel.js | Service.js M

Backend > Model > MessageModel.js > ...

```
1   const mongoose = require('mongoose');
2
3   const MessageSchema = new mongoose.Schema({
4     subject: { type: String, required: true, trim: true, minlength: 3, maxlength: 100 },
5     sender: { type: String, required: true, trim: true, minlength: 2, maxlength: 50 },
6     recipient: { type: String, required: true, trim: true, minlength: 2, maxlength: 50 },
7     // New: store actual user ids to route by login
8     senderId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
9     recipientId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
10    message: { type: String, required: true, trim: true, minlength: 1, maxlength: 1000 },
11    date: { type: Date, default: Date.now },
12    status: { type: String, enum: ['Unread', 'Read', 'Archived'], default: 'Unread' },
13    // Read tracking
14    isRead: { type: Boolean, default: false },
15    readAt: { type: Date, default: null }
16  });
17
18  module.exports = mongoose.model('Message', MessageSchema);
19
```

**Finance model**



```
JS FinanceModel.js ×

Backend > Model > JS FinanceModel.js > ...

1   const mongoose = require("mongoose");
2   const Schema = mongoose.Schema;
3
4   const financeSchema = new Schema({
5       description: { type: String, required: true },
6       Project_Name: { type: String, required: true },
7       category: { type: String, required: true },
8       amount: { type: Number, required: true },
9       date: { type: Date, default: Date.now },
10      status: { type: String, required: true, default: "Unpaid" },
11      project: { type: Schema.Types.ObjectId, ref: "Project" },
12      bankSlipPath: { type: String },
13      stage: {
14          type: String,
15          enum: ["deposit", "balance", "full"],
16          default: "full"
17      },
18      expectedAmount: { type: Number },
19      paidAmount: { type: Number },
20      paidAt: { type: Date },
21      submittedBy: { type: Schema.Types.ObjectId, ref: "User" },
22      source: {
23          type: String,
24          enum: ["admin", "client", "system"],
25          default: "admin"
26      }
27  });
28
29  module.exports = mongoose.model("FinanceModel", financeSchema)
```